

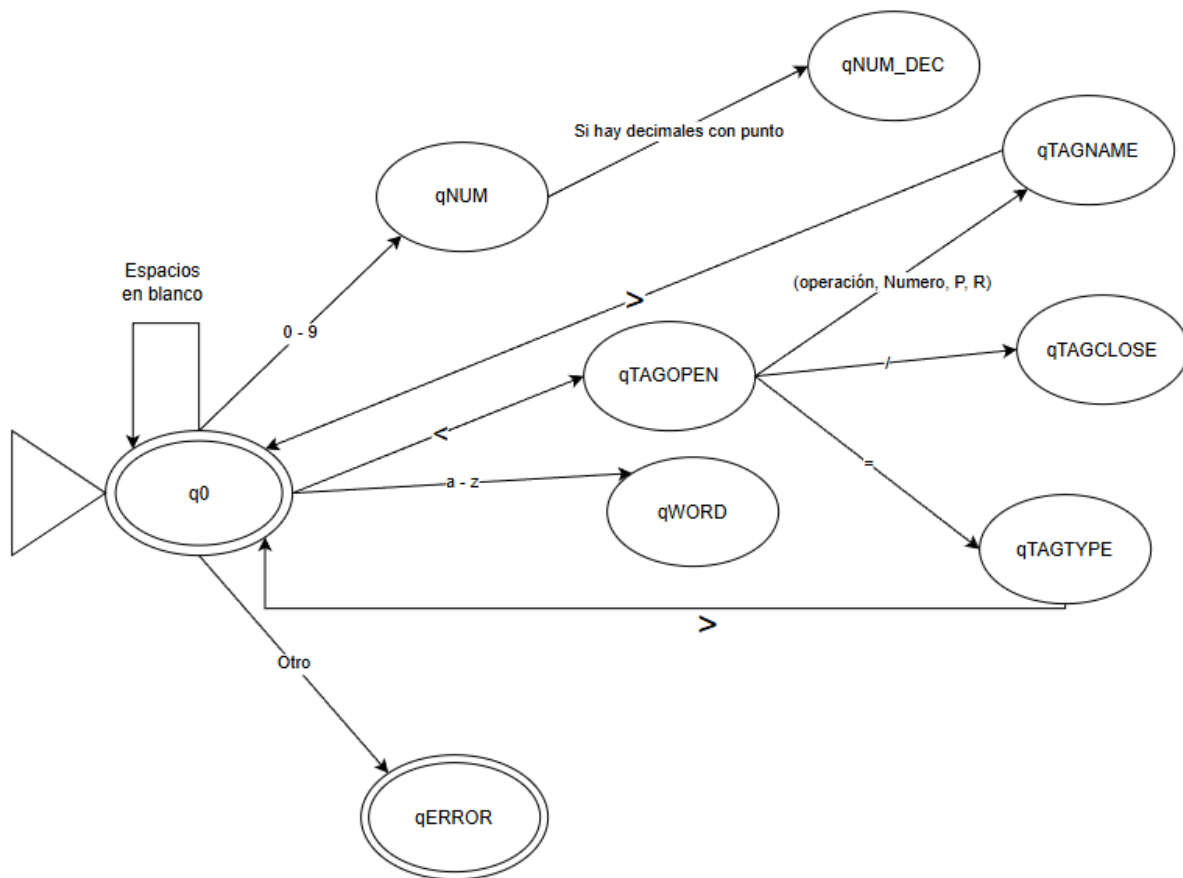
Universidad Rafael Landívar
Facultad de Ingeniería
Ingeniería en Informática Y Sistemas
Lenguajes Formales Y Autómatas
Catedrático: Ing. Damaris Campos

Proyecto NO.2 (Fase 1)
“Entrega de expresiones de la fase 1”

Pedro Pablo Cossío Rivas-1054723
Rodrigo José Ruiz Juarez-1037623

Primero se mostrarán los diagramas de manera general es decir cómo es que fueron planteados y posteriormente se demostrara cómo funciona las tablas de transición, el DFA y la tabla Follow para que sea un poco más entendible la ejecución de lo que pasa con ejemplos.

DFA



Estados principales de DFA

Estado	Descripción	Transición
q0	Estado inicial.	Según el primer carácter:
	Si es < → qTAGOPEN (inicio etiqueta)	
	Si es dígito → qNUM	
	Si es letra → qWORD	
	Si es espacio → quedarse en q0	
	Otro → qERROR	
qNUM	Reconoce números (0-9 y opcionalmente. y más dígitos).	Si aparece. qNUM_DEC; si no, acepta como NUMERO.
qNUM_DEC	Después del punto decimal.	Acepta dígitos qNUM_DEC; si no hay más acepta NUMERO.
qWORD	Secuencia de letras o guiones bajos.	Continúa mientras haya letras; al final acepta IDENTIFICADOR.
qTAGOPEN	Se leyó <.	Si sigue / qTAGCLOSE (etiqueta de cierre). Si sigue letra → qTAGNAME (etiqueta abierta).
qTAGNAME	Leyendo nombre de etiqueta (operación, Numero, P, R).	Si aparece = qTAGTYPE (atributo de operación). Si aparece > q0 (fin de etiqueta).
qTAGTYPE	Después del = se espera una operación (SUMA, RESTA, etc.).	Lee letras > qTAGTYPE; si aparece > acepta ETIQUETA ABIERTA.
qTAGCLOSE	Después de </.	Lee letras > qTAGCLOSE; si aparece > acepta ETIQUETA CERRADA.
qERROR	Cualquier carácter no reconocido.	Genera token ERROR y vuelve a q0.

Se realizó la tabla del DFA por si no es tan entendible como esta echo.

Tabla de transición

State	Input	Next State	Notes / Token
q0	espacios (space, tab, \n, \r)	q0	Inicio
q0	<	qTAGOPEN	Empieza las etiquetas
q0	</ (secuencia)	qTAGCLOSE	Cierran las etiquetas (handled as '<' then '/')
q0	[0-9]	qNUM	Empieza el numero de token
q0	[A-Za-z] or ' _ '	qWORD	Se indentifica/ palabras
q0	0	q0	Se emite un IGUAL token (un caracter-char)
q0	>	q0	Se emite un Mayor token (un caracter-char)
q0	ninguno/ desconocido	qERROR	No se reconoce el token > ERROR token
qNUM	[0-9]	qNUM	Se integran las partes
qNUM	.	qNUM_DEC	Parte decimal → Parte fraccional
qNUM	letra or ' _ '	qWORD	Fallback: se tomo como identificado (raro)
qNUM	otro (no-digito, non-'.')	q0	Se acepta NUMERO de token y se procesa char
qNUM_DEC	[0-9]	qNUM_DEC	Se consumen digitos fraccionales
qNUM_DEC	otro (no-digit)	q0	Se acepta NUMERO de token y se procesa char
qWORD	[A-Za-z] or ' _ '	qWORD	Se consume el identificador
qWORD	[0-9]	qWORD	El identificador puede llegar a contener digitos despues
qWORD	other (non-letter/digit/ _)	q0	Se acepta Identificador de token y se procesa char
qTAGOPEN	/	qTAGCLOSE	Si '/' luego '<', se pone el tag de cerrado
qTAGOPEN	[A-Za-z] or ' _ '	qTAGNAME	Empezar etiqueta (e.g., Operacion, Numero, P, R)
qTAGOPEN	Otro	qERROR	tag mal echa inicio
qTAGNAME	[A-Za-z] or ' _ '	qTAGNAME	continua tag name
qTAGNAME	0	qTAGTYPE	Se asigna un atributo (e.g., Operacion=SUMA)
qTAGNAME	>	q0	Cierra tag→ accept ETIQUETA_ABIERTA

qTAGNAME	Espacios	qTAGNAME	Permite espacios en blanco '>' or '='
qTAGTYPE	[A-Za-z] or '_'	qTAGTYPE	Consume las operaciones(SUMA, RESTA...)
qTAGTYPE	Espacios	qTAGTYPE	Permite espacios en blanco
qTAGTYPE	>	q0	Cierra tag > accept ETIQUETA_ABIERTA (with type)
qTAGTYPE	Otro	qERROR	Caraacter invalido inside tag type
qTAGCLOSE	[A-Za-z] or '_'	qTAGCLOSE	Consume tag de cerrado name
qTAGCLOSE	>	q0	Close tag accept ETIQUETA_CERRADA
qTAGCLOSE	Otro	qERROR	Malformed closing tag
qERROR	ninguno	q0	EmiteERROR token y regresa a q0

Tabla Follow

Hoja (No terminal)	Etiqueta (FIRST)	FOLLOW (conjunto)
S (inicio)	{ OPEN_OP, ϵ }	{ \$ }
OperList	{ OPEN_OP, ϵ }	{ \$ }
Operacion	{ OPEN_OP }	{ OPEN_OP, OPEN_NUM, CLOSE_OP, \$ }
OperConten t	{ OPEN_P, OPEN_R, OPEN_NUM, OPEN_OP, ϵ }	{ CLOSE_OP }
P_opt	{ OPEN_P, ϵ }	{ OPEN_R, OPEN_NUM, OPEN_OP, CLOSE_OP }
R_opt	{ OPEN_R, ϵ }	{ OPEN_NUM, OPEN_OP, CLOSE_OP }
Operandos	{ OPEN_NUM, OPEN_OP, ϵ }	{ CLOSE_OP }
Operando	{ OPEN_NUM, OPEN_OP }	{ OPEN_NUM, OPEN_OP, CLOSE_OP }
Numero	{ OPEN_NUM }	{ OPEN_NUM, OPEN_OP, CLOSE_OP }
P	{ OPEN_P }	{ OPEN_R, OPEN_NUM, OPEN_OP, CLOSE_OP }
R	{ OPEN_R }	{ OPEN_NUM, OPEN_OP, CLOSE_OP }

Árbol de expresión

Primero trataremos de analizar una operación normal

<Operacion=Suma>

<Numero>5</Numero>

<Operacion=Multiplicacion>

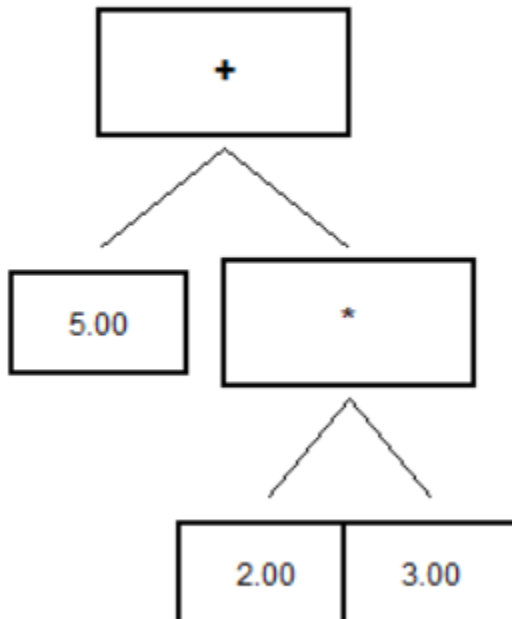
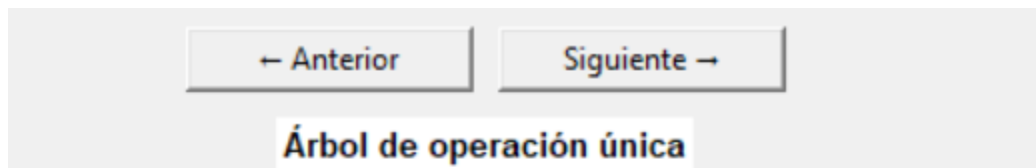
<Numero>2</Numero>

<Numero>3</Numero>

</Operacion>

</Operacion>

El árbol de operaciones se muestra de la siguiente forma



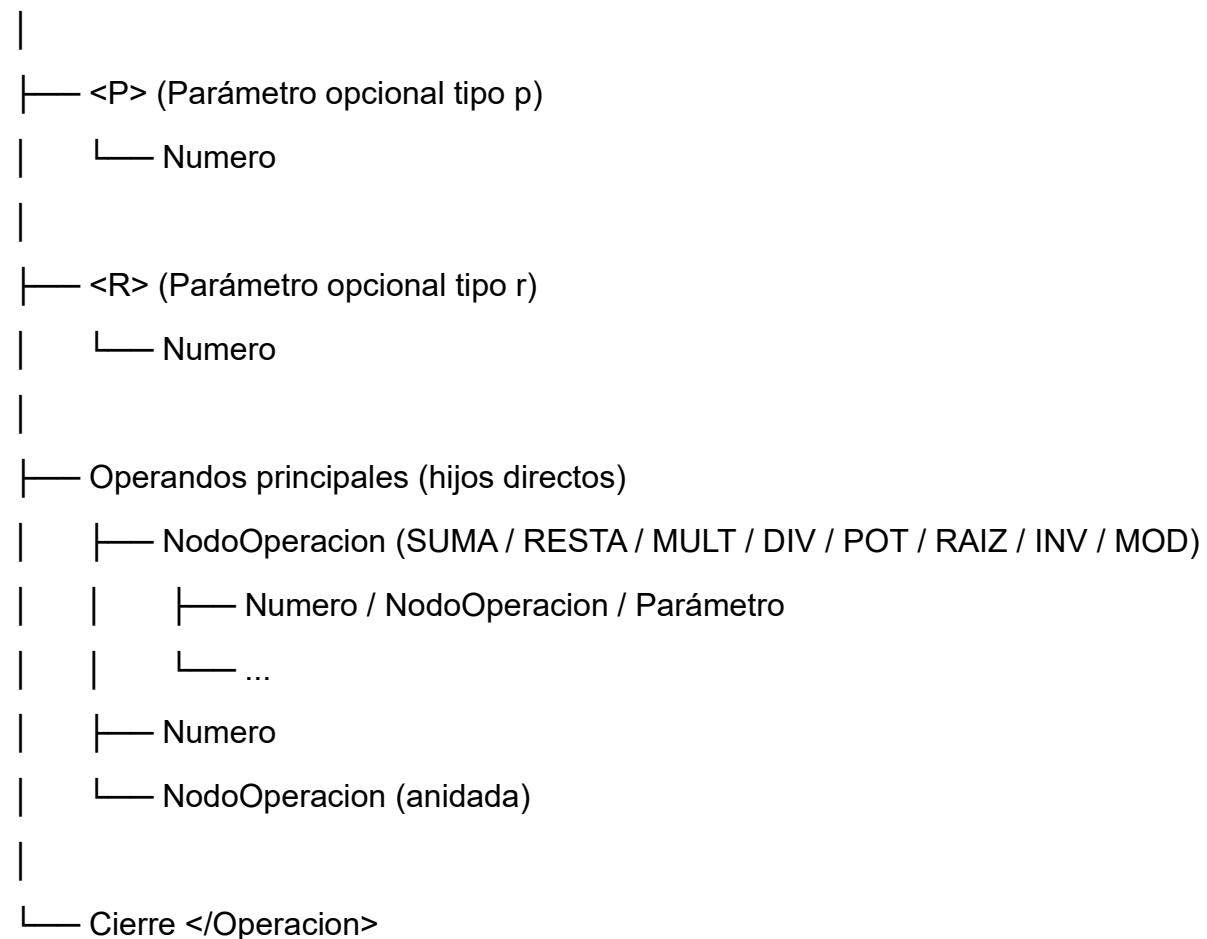
De esta forma obtenemos la expresión y los resultados que deberían de dar

1. $(5+(2*3)) = 11.000$

De esta forma podemos llegar a ver cómo funciona el analizador léxico y cómo funciona el árbol de expresión.

En este caso el árbol de funciones que se codifica tiene este ejemplo de manera general en este caso la conceptualización de los árboles es de la siguiente forma

NodoOperacion (raíz)



Estructura jerárquica de las operaciones generales

Operacion

└─ P (parámetro opcional)

| └─ Numero

└─ R (parámetro opcional)

| └─ Numero

└─ Numero / Operacion / P / R (n operandos)

└─ /Operacion

Cálculo de funciones

Dentro del calculo de funciones existe un evaluador para poder recorrerlo de manera recursiva.

Tipo de operación	Símbolo	Cálculo interno (recursivo)
SUMA	+	Suma todos los operandos: Σ operandos
RESTA	-	Resta en orden: $a - b - c$
MULTIPLICACION	*	Multiplica todos los operandos: Π operandos
DIVISION	/	Divide secuencialmente: $a / b / c$
POTENCIA	^	Usa 2 operandos: base ^ exponente
RAIZ	r	Usa 2 operandos: $\text{radicando}^{(1/\text{índice})}$
INVERSO	i	$1 / \text{valor}$
MOD	%	Módulo entre operandos: $a \% b \% c$

En este caso el evaluador se aplica de manera funcional

evaluar(nodo):

si nodo es número → devolver su valor

si nodo es operación:

evaluar recursivamente cada operando

aplicar la función correspondiente al tipo

devolver el resultado